Temporización y máquinas de estados: atender más de una tarea

Construir una secuencia que siga atendiendo eventos mientras transcurre un intervalo.

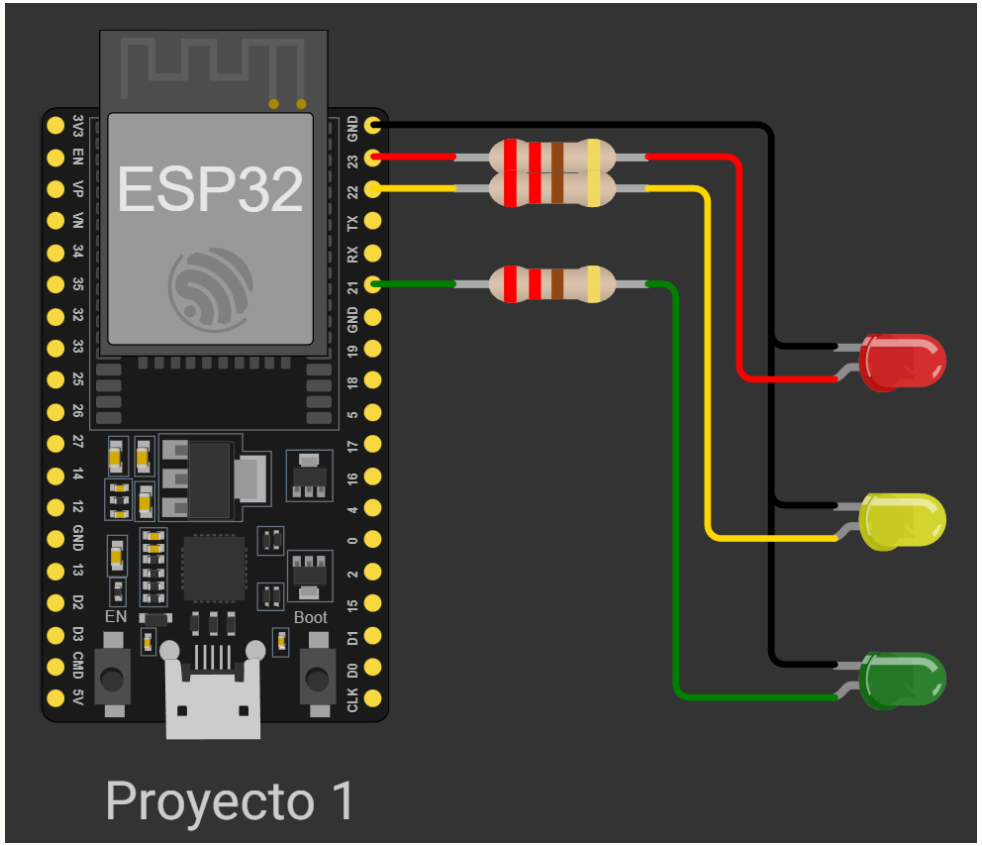
La adquisición de sensores, los avisos y la interfaz de un banco UAV necesitan convivir sin esperas largas.

EN LÍNEA 2 HORAS ESP32 CLÁSICO

Por qué un programa correcto puede responder tarde

delay() expresa una espera cómoda cuando solo importa una secuencia. Mientras esa función bloquea el recorrido principal, no se ejecutan las otras instrucciones de loop(). Algunos periféricos e interrupciones pueden seguir funcionando, pero la lógica que dejaste después de la espera debe aguardar. Si un aviso requiere observar el botón durante una secuencia de tres segundos, una espera larga puede retrasar esa observación. El problema se descubre definiendo una tarea adicional, no prohibiendo una función por su nombre.

Por qué un programa correcto puede responder tarde



¿Qué condición cambia el estado? Localiza la entrada y la salida indicadora.

MECRO, proyecto 01. Esquema original del laboratorio para reconocer la arquitectura en Wokwi. Para montaje físico se aplica la tabla y las correcciones de esta lectura.

Interpretación · Por qué un programa correcto puede responder tarde

El semáforo del laboratorio ya usa millis() y switch. Lo conservamos y explicamos antes de aumentar su complejidad. Caja fuerte, detector de intrusos y ascensor ofrecen variantes de eventos y estados. En una misma clase no construiremos cuatro mecanismos: compararemos sus lógicas y cada equipo implementará una sola variación. Lo aprendido debe transferirse a un nuevo caso en lugar de convertirse en una lista de programas copiados.

Tiempo transcurrido y retorno del contador

millis() informa un contador de milisegundos desde el inicio. Para decidir si venció un intervalo, calculamos la resta entre el valor actual y el guardado. Con un tipo sin signo de 32 bits, esa resta permite tratar correctamente un retorno del contador para los intervalos cortos del curso. La comparación directa contra un instante futuro calculado con suma puede comportarse mal al atravesar ese retorno. No se trata de memorizar una frase: se puede comprobar con números pequeños y después con valores cercanos al límite.

Patrón temporal

```
uint32_t now = millis();
if (uint32_t(now-last) >= interval_ms) {
  last = now;
  actualizarTarea();
}
```

Lee el fragmento. Predice una salida y cambia una sola condición antes de ejecutar.

Tiempo transcurrido y retorno del contador

$$\Delta t = \text{uint32}(t_{ahora} - t_{anterior})$$

Guardar last = now evita intentar recuperar todas las ejecuciones perdidas después de una demora. Actualizar last += interval_ms responde a otra política de agenda. Primero elegiremos una tarea y una política explícitas. Para medir RPM más adelante, usaremos el tiempo realmente transcurrido; para un modelo matemático de paso fijo, el tiempo del modelo tendrá su propio significado. Dos relojes con la misma unidad pueden representar referencias distintas.

Todos los tiempos de esta comparación usan el mismo tipo y la misma unidad.

Estado, evento y transición

Un estado conserva qué situación del sistema está activa. Un evento señala algo que ocurrió o una condición que acaba de cumplirse. La transición define a qué estado se pasa y qué acciones se ejecutan. En el semáforo, ROJO, VERDE y AMARILLO son estados. “Han transcurrido tres segundos” es una condición de transición. Una solicitud peatonal puede quedar guardada para atenderse cuando corresponda. Separar estas ideas permite dibujar la lógica antes de escribir switch.

Fragmento de la secuencia

Estado	Condición	Siguiente estado
ROJO	3 s transcurridos	VERDE
VERDE	3 s transcurridos	AMARILLO
AMARILLO	1 s transcurrido	ROJO

Solicitud durante verde

Se pulsa el botón durante verde. ¿Dónde conservas la solicitud hasta llegar a rojo?

En una variable de solicitud persistente. El evento no se conserva si solo se pregunta por el nivel instantáneo al llegar a rojo. La variable se limpia después de atender la condición definida.

Lectura de una transición pequeña

La función enter() centraliza la asignación del estado, la referencia temporal y las salidas. Así no quedan luces con valores heredados de otra rama. El programa de referencia registra la solicitud y no obliga a que el usuario siga pulsando hasta el momento de atención. En una interfaz real se puede añadir el antirrebote de la clase anterior. Ambas funciones tienen propósitos distintos: una define la secuencia y la otra interpreta el contacto.

Núcleo de c03_estados

```
if (digitalRead(BTN) == LOW) request = true;
if (state == State::GREEN
    && uint32_t(now-since) >= 3000) {
  enter(State::YELLOW, now);
}
```

Lee el fragmento. Predice una salida y cambia una sola condición antes de ejecutar.

Interpretación · Lectura de una transición pequeña

Para depurar, construye una tabla con tiempo, estado, entrada y acción. No es necesario imprimir miles de mensajes por segundo. Registra transiciones y hechos importantes para que el monitor permita reconstruir la secuencia. Si cada recorrido de loop() emite una línea, la propia comunicación puede ocupar una parte importante del tiempo y esconder lo que intentabas observar. Un registro útil se diseña alrededor de preguntas concretas.

Laboratorio de respuesta y variación

La prueba busca una diferencia de comportamiento: por ejemplo, qué sucede con una solicitud breve que aparece durante una espera. Observar solo que las luces completan el ciclo no responde esa pregunta. Si el equipo afirma que la respuesta es “inmediata”, debe decir cómo la observó y qué demora podría no detectar con ese método. En esta etapa basta demostrar que el evento se conserva y se atiende según una regla.

Registro de prueba

Tiempo / ms	Estado	Evento / acción
Inicio	Por registrar	Estado inicial
Solicitud	Por registrar	Entrada observada
Transición	Por registrar	Cambio de salidas
Atención	Por registrar	Solicitud consumida

Práctica · Laboratorio de respuesta y variación

1. Dibuja la tabla de transiciones y señala qué variables deben conservarse.
2. Ejecuta el semáforo y pulsa en cada uno de sus tres estados.
3. Añade un contador de solicitudes atendidas sin detener la secuencia.
4. Compara una versión con delay(3000) y otra por tiempo transcurrido.
5. Introduce una acción de parpadeo independiente para mostrar dos agendas.
6. Elige caja fuerte o PIR como variante y describe su estado inicial y su transición de retorno.
7. Conserva un registro de eventos que permita explicar una prueba completa.

Tres estados para un banco de sensores

Una aplicación cercana al mercado UAV puede usar INICIO, LISTO y FALLO. En INICIO se preparan los periféricos; en LISTO se adquieren y muestran datos; en FALLO se informa una condición que impide tratarlos como recientes. El estado describe el sistema, mientras un LED u OLED lo presenta. La misma estructura permitirá que la clase final detecte un sensor ausente sin seguir mostrando una lectura antigua como si fuera nueva.

Transferencia de la lógica

Proyecto	Evento	Qué se conserva
Caja fuerte	Clave aceptada	Estado de acceso
PIR	Cambio de movimiento	Estado de aviso
Banco UAV	Lectura vencida	Estado de datos

Interpretación · Tres estados para un banco de sensores

Investiga en una biblioteca si sus funciones esperan a terminar una operación. Una interfaz aparentemente sencilla puede ocultar una espera. Identifica el comportamiento de motor.step() en la variante del ascensor y de pulseIn() en el sensor ultrasónico. Una arquitectura sin delay() todavía puede bloquear en otras llamadas. La habilidad es reconocer la duración y el lugar de la espera, no buscar una palabra prohibida.

Práctica guiada

1. Plantear una solicitud que ocurra en mitad de una secuencia.

2. Describir la tabla de estados antes de ejecutar el programa.

3. Comparar las dos políticas de temporización con c03_estados.

4. Transferir la misma lógica a INICIO, LISTO y FALLO.

Aplicación en un banco para drones

La adquisición de sensores, los avisos y la interfaz de un banco UAV necesitan convivir sin esperas largas.

Explica una decisión con el esquema, el programa o un dato. ¿Qué queda sin comprobar?

Preguntas de cierre

1. ¿Quitar delay() garantiza un programa no bloqueante?
2. ¿Qué guarda la solicitud?
3. ¿posicionActual demuestra posición física?

Respuestas y razonamiento

1. No; otras llamadas también pueden esperar. Hay que estudiar su duración y su efecto.

2. Una variable persistente que se limpia al atenderla según la regla.

3. Solo si existe una medición que la respalde; un conteo de pasos ordenados puede ser únicamente una estimación.

Entrega de la clase

Tabla de estados, variante temporal y registro de atención de eventos.

Identifica versión, unidad y procedencia. Conserva la evidencia y explica qué probarías después.

ADC y calibración: de una lectura a una decisión

Interpretar lecturas analógicas, construir una escala de trabajo y justificar dos umbrales de histéresis.

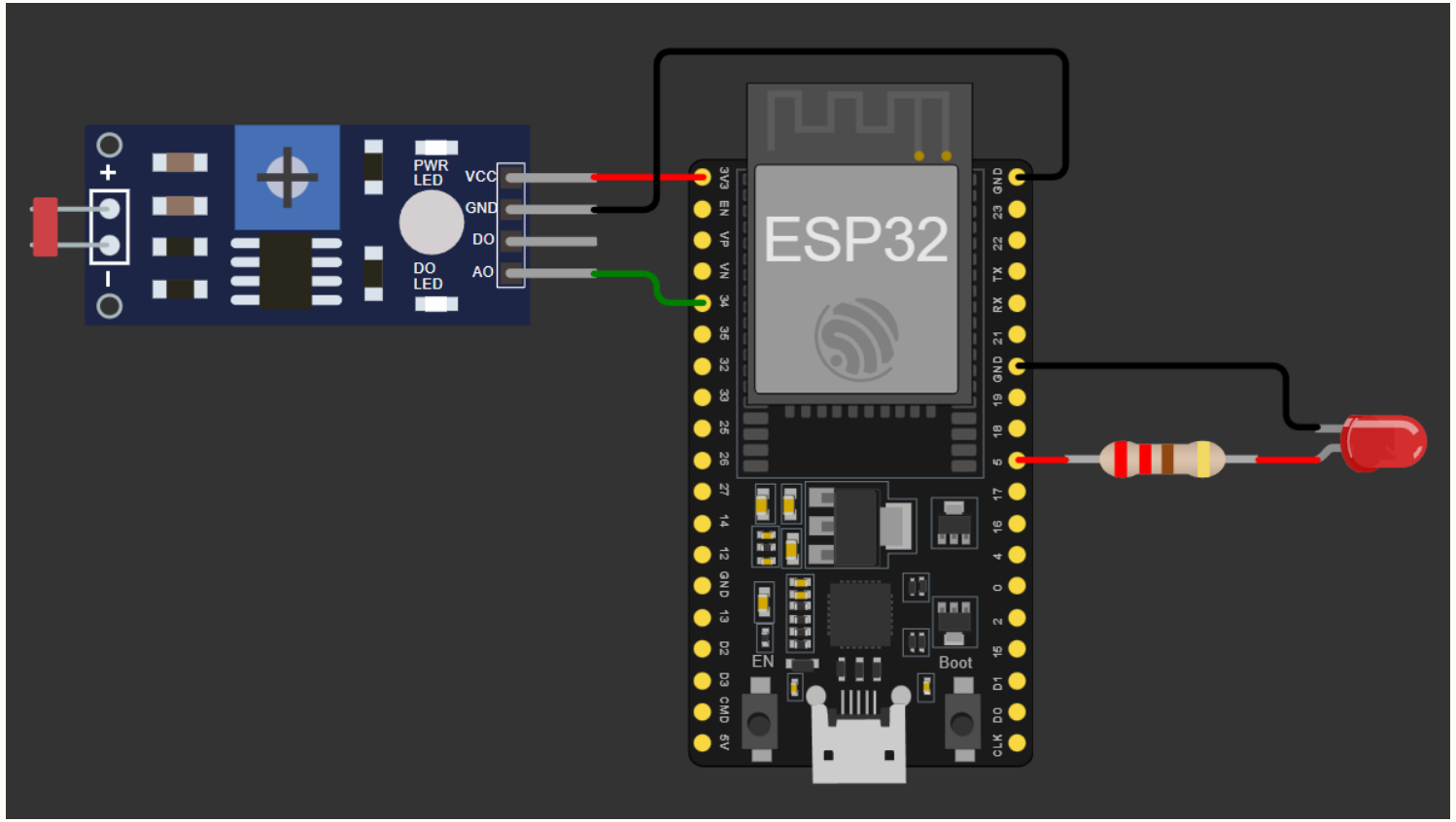
Un sensor de batería, luz o posición en un sistema UAV requiere distinguir lectura digital, magnitud física y validez del dato.

EN LÍNEA 2 HORAS ESP32 CLÁSICO

Qué convierte un ADC

Un sensor analógico representa una condición mediante una tensión. El convertidor analógico a digital devuelve un número que depende de esa tensión y de la configuración. En una lectura de 12 bits existen 4096 códigos posibles, numerados de 0 a 4095. Esa cantidad describe resolución de código; no demuestra exactitud ni una relación perfectamente lineal en toda la entrada. El ESP32 clásico ofrece varios canales y restricciones que hay que relacionar con el pin elegido.

Qué convierte un ADC



¿Qué nodo entrega una señal analógica? Distingue alimentación y lectura.

MECRO, proyecto 03. Esquema original del laboratorio para reconocer la arquitectura en Wokwi. Para montaje físico se aplica la tabla y las correcciones de esta lectura.

Interpretación · Qué convierte un ADC

GPIO34 pertenece a ADC1 y es una entrada sin pull-up interno. En nuestro banco se conecta a un potenciómetro o a la salida analógica de un divisor de luz que respete el rango del pin. Elegimos ADC1 para que la práctica se pueda conservar cuando añadamos Wi-Fi. La elección no convierte el pin en tolerante a 5 V. La configuración de atenuación y el rango útil se consultan para la variante real.

La LDR cambia su resistencia con la iluminación. El divisor la transforma en tensión. Si se intercambia la posición de la LDR y del resistor fijo, se invierte la tendencia de la lectura. Por eso no se enseña que oscuridad equivale siempre a un número pequeño. Se sigue el circuito y se mide la tendencia. Ese razonamiento también sirve al interpretar un joystick o cualquier sensor con salida analógica.

La escala ideal y la escala calibrada

En un modelo ideal con escala completa de 3,0 V, el código 2048 representa aproximadamente 1,50 V. En hardware comparamos esa idea con una medición y con la API calibrada disponible. analogRead() devuelve código bruto y analogReadMilliVolts() ofrece una conversión calibrada según el soporte de la plataforma. Una variable llamada voltaje no queda calibrada por recibir ese nombre. El alumno conserva el método de obtención junto al dato.

Cuatro conceptos distintos

Concepto	Qué describe	Ejemplo de comprobación
Resolución	Cantidad de códigos	12 bits: 4096 códigos
Exactitud	Cercanía a referencia	Comparar con instrumento
Repetibilidad	Dispersión al repetir	Varias lecturas iguales
Saturación	Extremo de conversión	Más entrada, mismo código

La escala ideal y la escala calibrada

$$\hat{V} = \frac{N}{4095} V_{FS}$$

Para construir una escala de trabajo, observa dos condiciones reproducibles. En la LDR pueden ser dos niveles de iluminación del banco; en un joystick, centro y recorridos permitidos. Los valores se guardan como datos de ese montaje. No se presentan como propiedades universales del componente. Conviene evitar las zonas donde la lectura se satura y perdería capacidad para distinguir cambios.

Esta relación describe un ADC ideal con escala VFS definida. No declara el rango real ni la exactitud del ESP32.

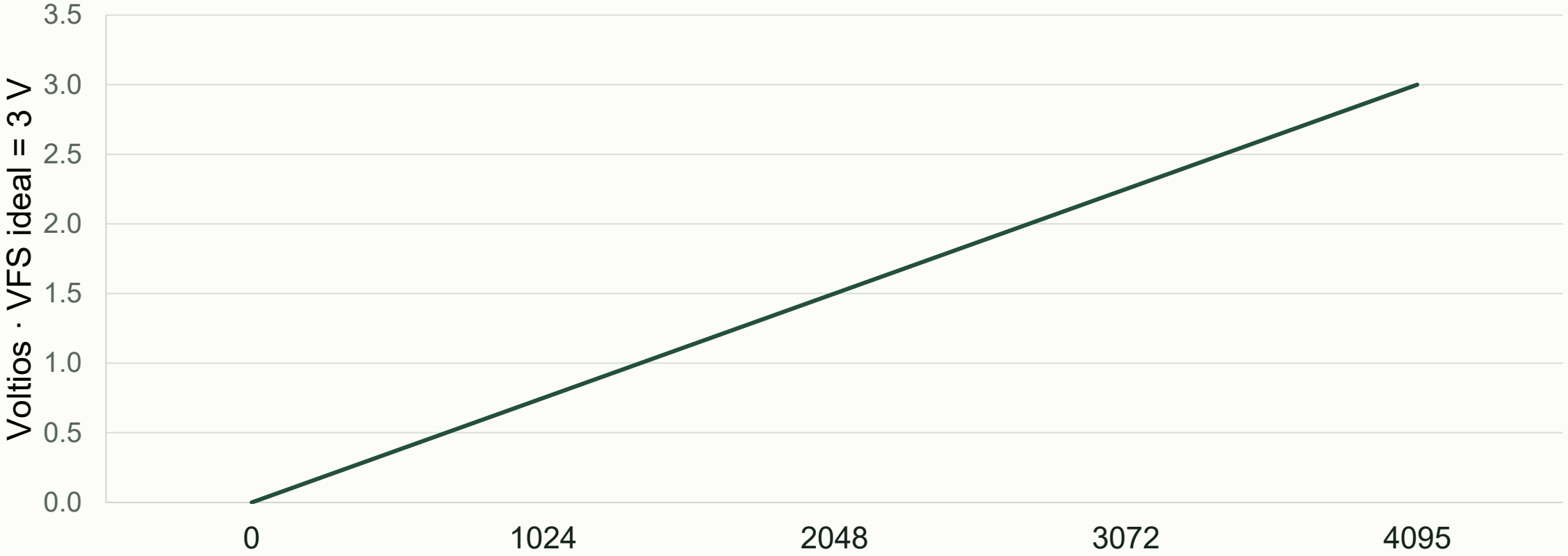
Ruido, promedio e histéresis

Una lectura real puede fluctuar aunque el objeto parezca quieto. Un promedio reduce algunas variaciones rápidas, pero también retrasa cambios y no corrige un sesgo de calibración. La histéresis usa dos límites para evitar que una salida cambie repetidamente cuando la entrada oscila cerca de una frontera. Si el sistema ya está encendido, requiere superar el umbral de apagado para cambiar de estado; entre ambos conserva su decisión.

Operaciones sobre la lectura

Operación	Ayuda a	No resuelve por sí sola
Promedio	Atenuar fluctuación	Calibración absoluta
Histéresis	Evitar conmutación repetida	Cambiar polaridad del divisor
Límite de rango	Detectar o recortar extremos	Recuperar información saturada

Referencia ideal del ejercicio



Modelo ideal para aprender la escala. No representa calibración ni exactitud del ADC físico.

Código que conserva una decisión

Entre ambos umbrales no se asigna un valor nuevo a on. Esa ausencia es deliberada: la variable mantiene el estado anterior. Cambiar else if por otra regla independiente puede modificar el comportamiento si las condiciones se solapan. La versión completa configura resolución y atenuación y registra tiempo, código y estado. El periodo de 50 ms se declara para que los datos puedan compararse.

Núcleo de c04_adc

```
int raw = analogRead(34);
if (raw < 1500) on = true;
else if (raw > 1800) on = false;
digitalWrite(23, on);
```

Lee el fragmento. Predice una salida y cambia una sola condición antes de ejecutar.

Interpretación · Código que conserva una decisión

map() realiza una transformación de escala con aritmética entera y no limita automáticamente la salida. Si un valor excede los extremos de entrada, el resultado puede exceder los extremos deseados. Cuando el proyecto necesita recortar, se hace explícitamente y se conserva el hecho de que la entrada salió del rango. Una escala de porcentaje es una convención de trabajo; no equivale automáticamente a una medida física certificada de iluminación o humedad.

Laboratorio con una referencia visible

La simulación permite controlar una entrada y estudiar la lógica del programa. El comportamiento analógico del mundo real añade dispersión, impedancias, alimentación y límites que el modelo no reproduce completamente. Una práctica de calibración física requiere una referencia y condiciones conservadas. El alumnado sin kit puede resolver la parte lógica y analizar un conjunto de datos identificado, mientras el docente demuestra la medición real.

Predicción y explicación

¿map() limita la salida?

Registrar una calibración pequeña y justificar dos umbrales.

Práctica · Laboratorio con una referencia visible

1. Verifica la alimentación y la referencia del divisor o potenciómetro.

2. Registra diez lecturas con la entrada quieta y anota mínimo, máximo y contexto.

3. Repite en dos condiciones distintas y determina el sentido de la respuesta.

4. Ejecuta c04_adc y sustituye los umbrales de ejercicio por una propuesta de tu montaje.

5. Introduce cambios lentos alrededor de la frontera y compara uno y dos umbrales.

6. Conserva los datos brutos junto a la salida; comprueba qué sucede al desconectar una entrada de forma controlada.

7. Explica una limitación del modelo de Wokwi frente al circuito físico.

Aplicación: observar batería sin conectar una batería al ADC

El principio de un divisor permite reducir una tensión para observarla, pero una batería real de dron exige un diseño de entrada completo y conocer sus límites. En este bloque practicamos con una fuente de baja tensión o un potenciómetro. El equipo puede dibujar una cadena conceptual de observación de batería: magnitud, acondicionamiento, ADC, calibración y umbral. El aprendizaje transferible consiste en justificar cada transformación y detectar qué falta por comprobar.

Predicción y explicación

¿Qué hace la histéresis entre límites?

Investiga qué ocurre con ADC2 del ESP32 clásico al usar Wi-Fi y por qué elegimos GPIO34. Localiza además qué expresa la atenuación. Presenta una lectura de la API en milivoltios con el método que la produjo, sin confundir el valor de alimentación de la placa con la escala exacta del convertidor. La conclusión de clase debe diferenciar una medida, una conversión ideal y una decisión de software.

Aplicación: observar batería sin conectar una batería al ADC

$$x_{norm} = \frac{x - x_{min}}{x_{max} - x_{min}}$$

Investiga qué ocurre con ADC2 del ESP32 clásico al usar Wi-Fi y por qué elegimos GPIO34. Localiza además qué expresa la atenuación. Presenta una lectura de la API en milivoltios con el método que la produjo, sin confundir el valor de alimentación de la placa con la escala exacta del convertidor. La conclusión de clase debe diferenciar una medida, una conversión ideal y una decisión de software.

Los extremos pertenecen a una calibración declarada; si coinciden, la transformación es inválida.

Práctica guiada

1. Mostrar que invertir el divisor invierte la tendencia de la lectura.
2. Distinguir códigos, voltios y porcentaje con un caso numérico.
3. Comprobar histéresis con la secuencia de cinco lecturas.
4. Registrar una calibración pequeña y justificar dos umbrales.

Aplicación en un banco para drones

Un sensor de batería, luz o posición en un sistema UAV requiere distinguir lectura digital, magnitud física y validez del dato.

Explica una decisión con el esquema, el programa o un dato. ¿Qué queda sin comprobar?

Preguntas de cierre

1. ¿12 bits significan 12 bits de exactitud?

2. ¿map() limita la salida?

3. ¿Qué hace la histéresis entre límites?

Respuestas y razonamiento

1. No. Describen códigos; la exactitud depende del sistema y de su calibración.

2. No; si el proyecto lo necesita, se limita explícitamente y se trata el fuera de rango.

3. Conserva la decisión anterior.

Entrega de la clase

CSV de lecturas, sentido del divisor y justificación de umbrales.

Identifica versión, unidad y procedencia. Conserva la evidencia y explica qué probarías después.